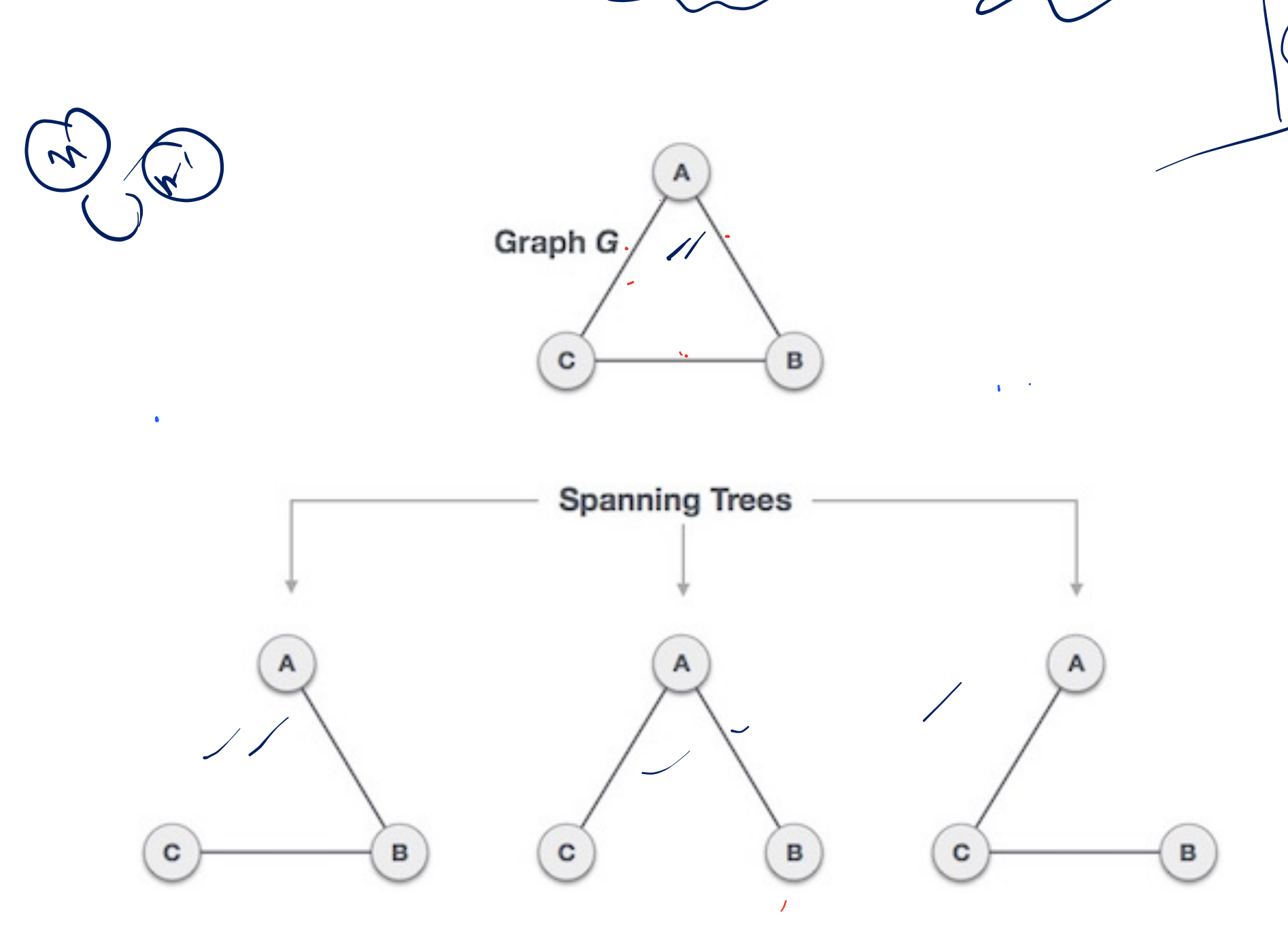
Lec-39
05/04/26 12:45 PM

Spanning Tree

A spanning tree is a subset of Graph G, which has all the vertices covered with minimum possible number of edges. Hence, a spanning tree does not have cycles and it cannot be disconnected..

180 km
Test → Bhai
150 MCQ



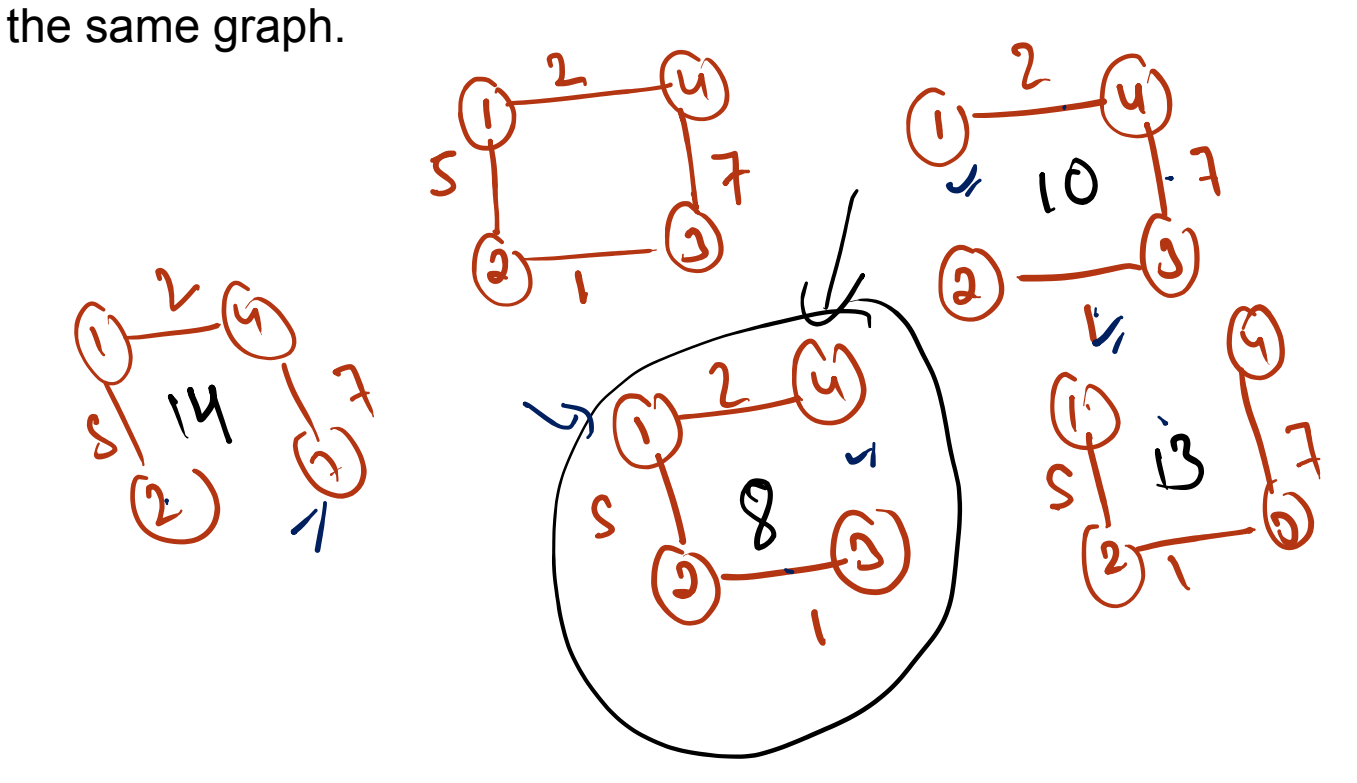
Properties of Spanning Tree

- A connected graph G can have more than one spanning tree.
- Spanning Tree has n-1 edges, where n is the number of Node (Vertices)
- All possible spanning trees of graph G, have the same number of edges and vertices.
- The spanning tree does not have any cycle (loops).
- Removing one edge from the spanning tree will make the graph disconnected, i.e. the spanning tree is minimally connected.
- Adding one edge to the spanning tree will create a circuit or loop, i.e. the spanning tree is maximally acyclic.

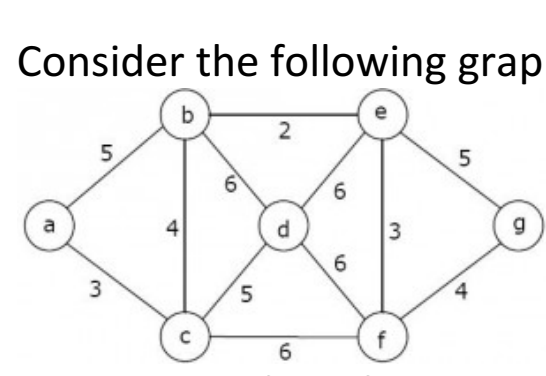
Minimum Spanning Tree (MST)
In a weighted graph, a minimum spanning tree is a spanning tree that has minimum weight than all other spanning trees of the same graph.

- Kruskal's Algorithm

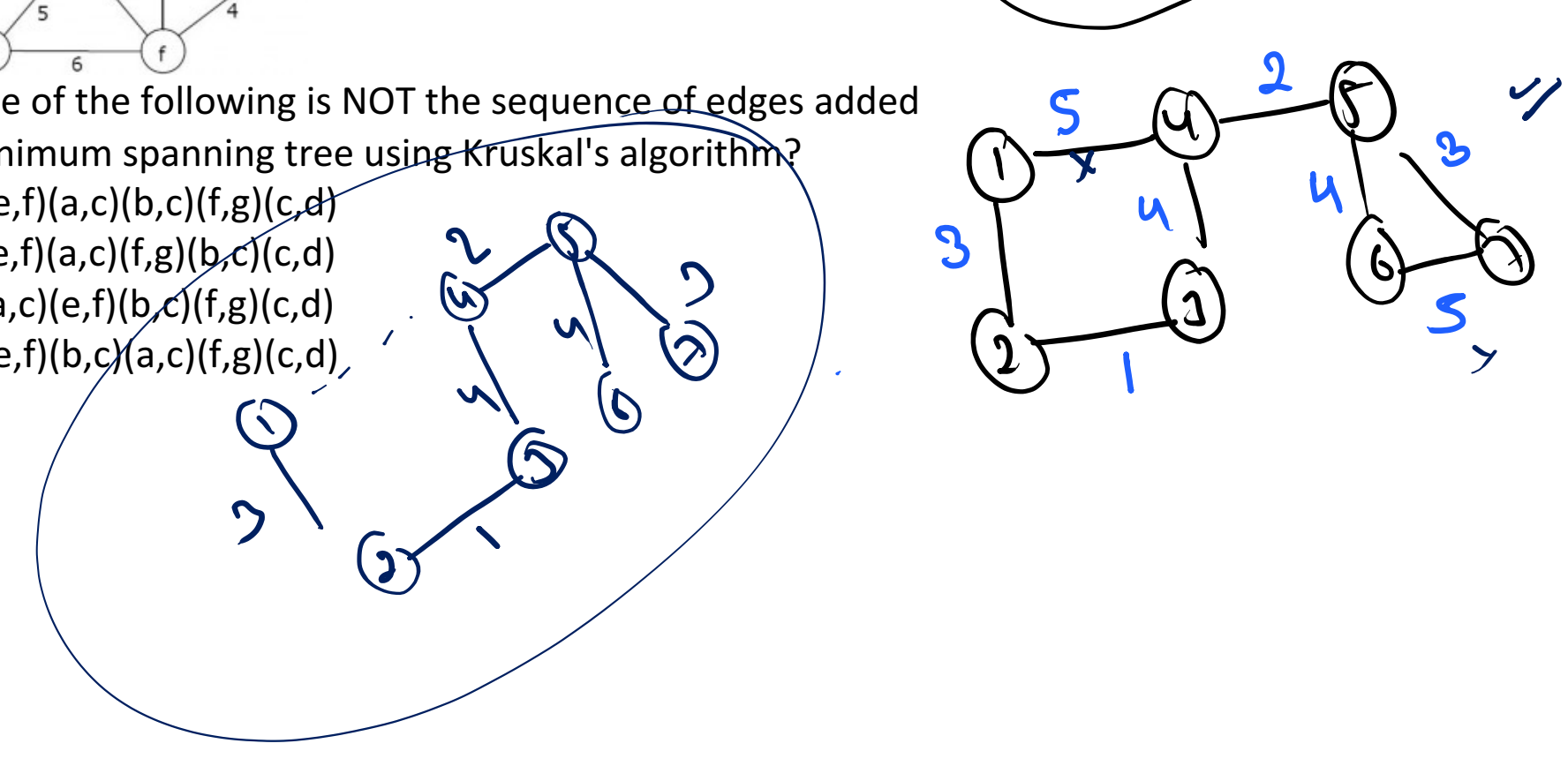
Prim's Algorithm
Both are greedy algorithms.



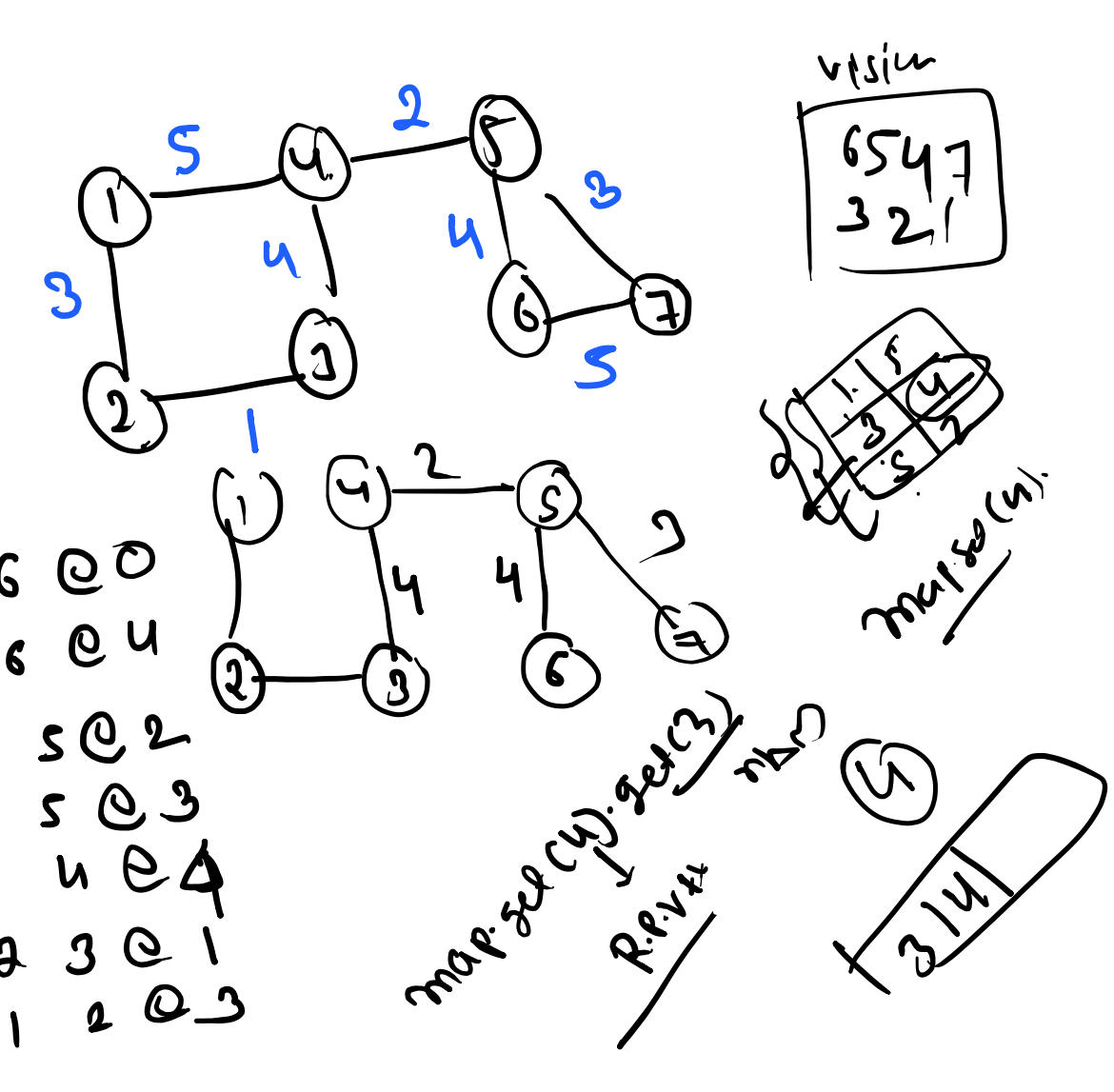
Consider the following graph:



Which one of the following is NOT the sequence of edges added to the minimum spanning tree using Kruskal's algorithm?
(A) (b,e)(e,f)(a,c)(b,c)(f,g)(c,d)
(B) (b,e)(e,f)(a,c)(f,g)(b,c)(c,d)
(C) (b,e)(a,c)(e,f)(b,c)(f,g)(c,d)
(D) (b,e)(e,f)(b,c)(a,c)(f,g)(c,d)



(V+E)

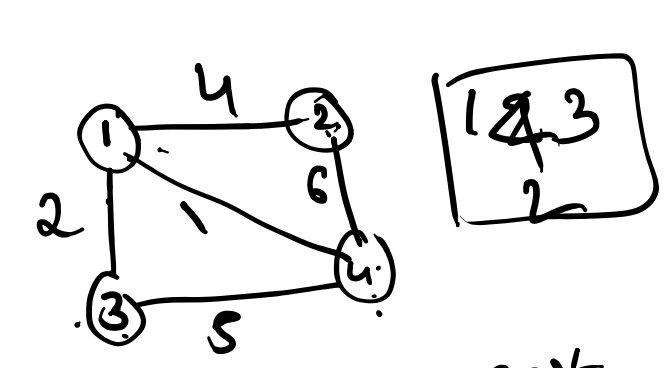


6 6@0
5 6@4
4 5@2
7 5@3
3 4@4
2 3@1
1 2@3

vtx	acq	cost
6	6	0
5	6	4
7	6	5
4	5	2
7	5	3
1	4	5
3	4	4
2	3	1
1	2	3

1. remove
2. ignore
3. marked visit
4. add nbrs
5. self

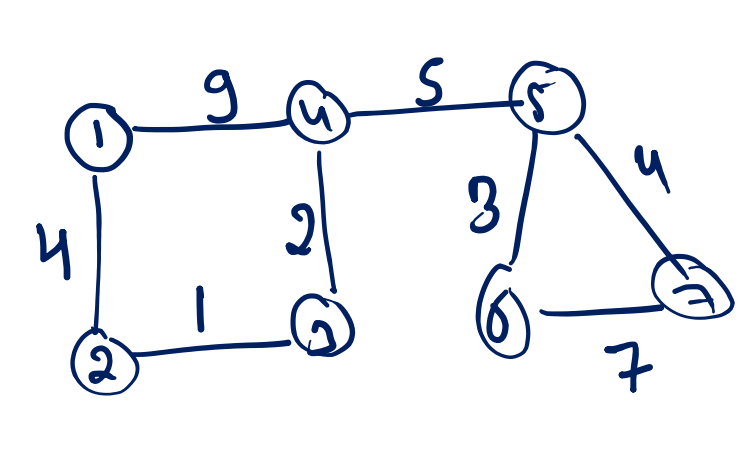
```java
public void PrimsAlgo() {
      PriorityQueue<PrimsPair> pq = new PriorityQueue<>();
      HashSet<Integer> visited = new HashSet<>();
      pq.add(new PrimsPair(1, 1, 0));
      while (!pq.isEmpty()) {
         // 1. remove
         PrimsPair rp = pq.poll();
         // 2. Ignore if Already visited
         if(visited.contains(rp.vtx)) {
            continue;
         }
         // 3. visited marked
         visited.add(rp.vtx);
         // 4. self work
         System.out.println(rp);
         // 5. add unvisited nbsr
         for(int nbrs:map.get(rp.vtx).keySet()) {
            if(!visited.contains(nbrs)) {
               int cost=map.get(rp.vtx).get(nbrs);
               pq.add(new PrimsPair(nbrs, rp.cost, cost));

            }
         }
      }
   }
```



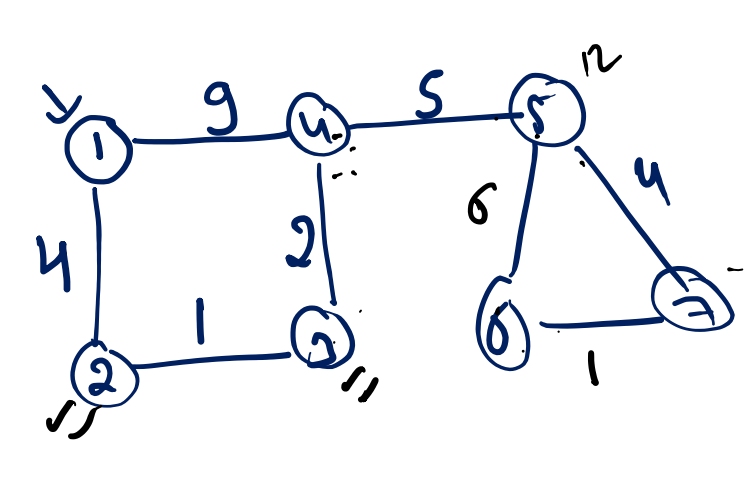
1 1@0
2 1@1
3 1@2
2 1@4

vtx	acq	cost
1	1	0
3	1	2
4	1	4
2	1	1
2	4	6
3	4	5

1→2 ✓
1→4→ 123
8
1→3
1→4
1→5
1→6
1→7



S.S.S.P
Dijkstra
Bellman Ford

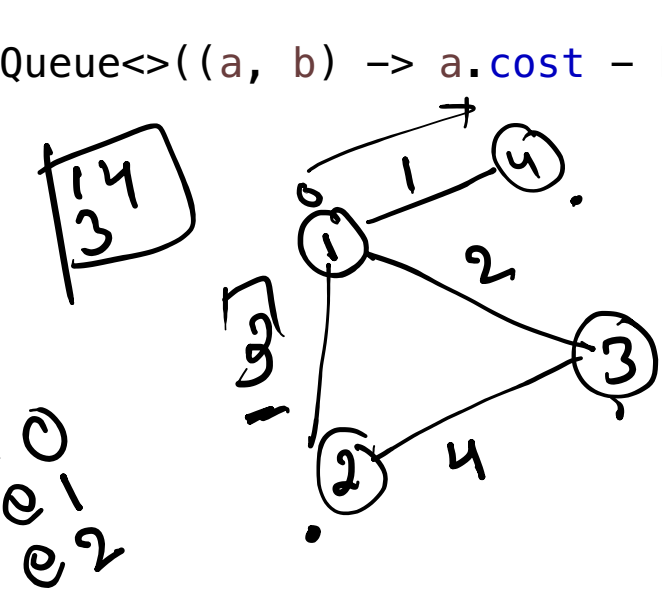


1234
576

1 1@0
2 12@4
3 123@5
4 1234@7
5 12345@12
7 123457@16
6 1234576@17

vtx	Path	cost
1	1	0
2	12	4
4	14	9
3	123	5
4	1234	7
5	12345	12
6	123456	18
7	123457	16
6	1234576	17

```java
public void Dijkstra(int src) {
      PriorityQueue<DijkstraPair> pq = new PriorityQueue<>((a, b) -> a.cost - b.cost);
      HashSet<Integer> visited = new HashSet<>();
      pq.add(new DijkstraPair(src, "" + src, 0));
      while (!pq.isEmpty()) {
         // 1. remove
         DijkstraPair rp = pq.poll();
         // 2. Ignore if Already visited
         if (visited.contains(rp.vtx)) {
            continue;
         }
         // 3. visited marked
         visited.add(rp.vtx);
         // 4. self work
          System.out.println(rp);
         // 5. add unvisited nbsr
         for (int nbrs : map.get(rp.vtx).keySet()) {
            if (!visited.contains(nbrs)) {
               int cost = map.get(rp.vtx).get(nbrs);
               pq.add(new DijkstraPair(nbrs,rp.acqpath+rp.vtx , rp.cost+cost));

            }
         }
      }

   }
```



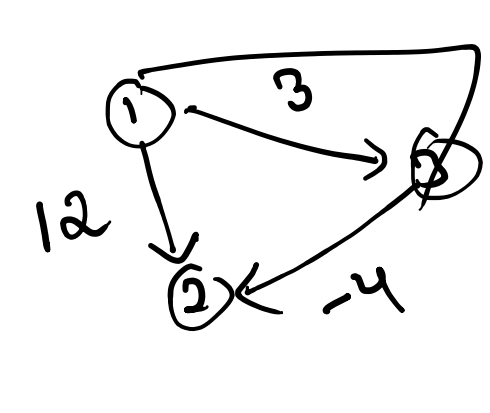
1 1@0
4 14@1
3 13@2
2 12@3

[2,3,4]

vtx	pth	cost
1	1	0
2	12	3
4	14	1
3	13	2
2	132	6

```java
public void Dijkstra(int src) {
      PriorityQueue<DijkstraPair> pq = new PriorityQueue<>((a, b) -> a.cost - b.cost);
      HashSet<Integer> visited = new HashSet<>();
      pq.add(new DijkstraPair(src, "" + src, 0));
      while (!pq.isEmpty()) {
         // 1. remove
         DijkstraPair rp = pq.poll();
         // 2. Ignore if Already visited
         if (visited.contains(rp.vtx)) {
            continue;
         }
         // 3. visited marked
         visited.add(rp.vtx);
         // 4. self work
          System.out.println(rp);
         // 5. add unvisited nbsr
         for (int nbrs : map.get(rp.vtx).keySet()) {
            if (!visited.contains(nbrs)) {
               int cost = map.get(rp.vtx).get(nbrs);
               pq.add(new DijkstraPair(nbrs,rp.acqpath+rp.vtx , rp.cost+cost));

            }
         }
      }

   }
```

123



1 1@0
2 12@2
3 13@3

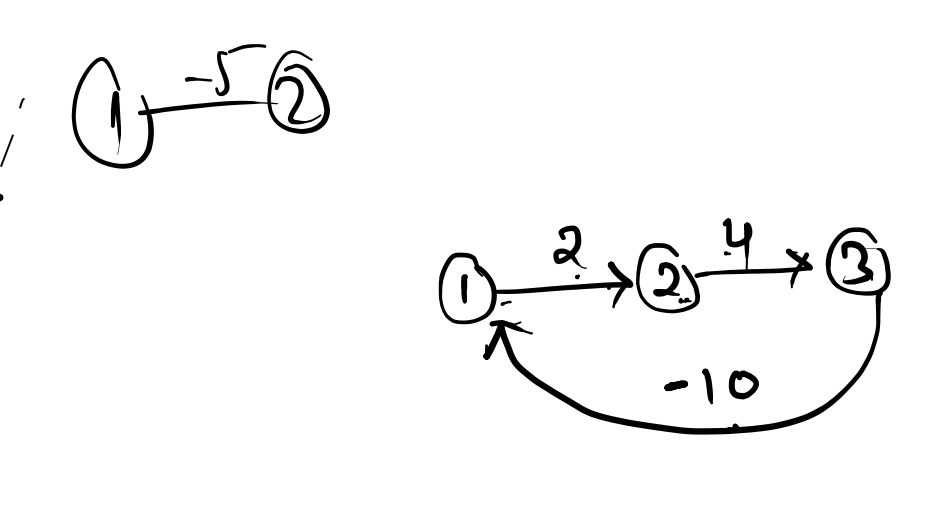
1	1	0
3	13	3
2	12	2

Dijkstra
dire

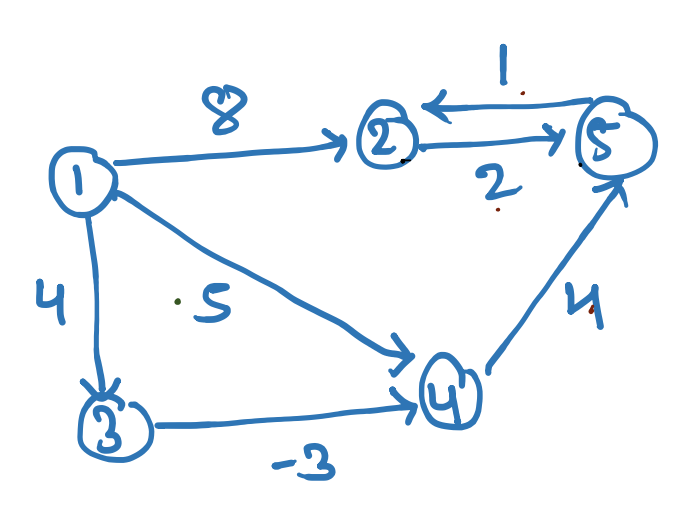
Dijkstra
undire

+v
-ve wt

① Bellman



1→2 →12 →2 ✓
1→3 → 123 →6
12312 → -2



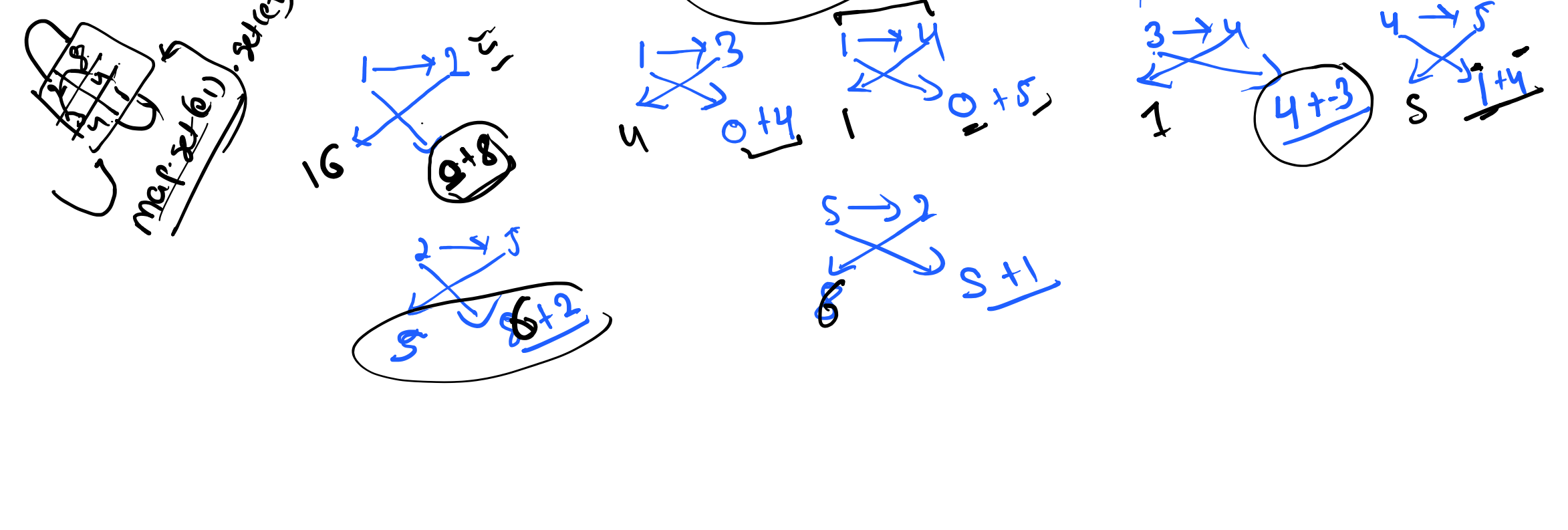
e1 e2 cost
1→2 8
1→3 4
1→4 5
3→4 -3
4→5 4
2→5 2
5→2 1

⑤ ④

⑥

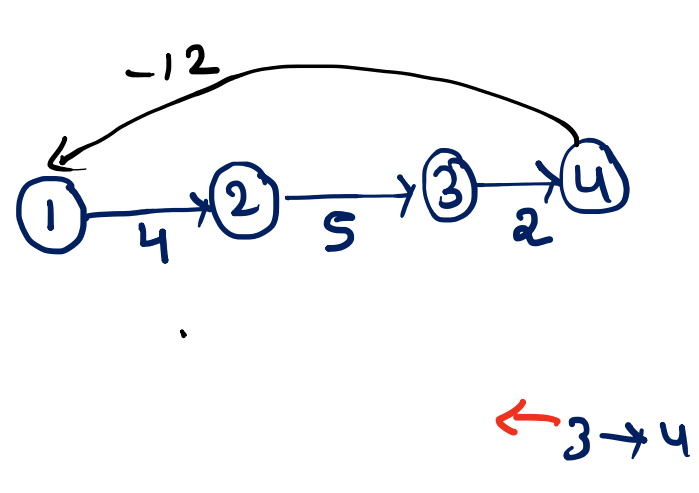
0	1	2	3	4	5
	0	6	4	1	5

dis(e.e2) > dis(e.e1) + e.cost

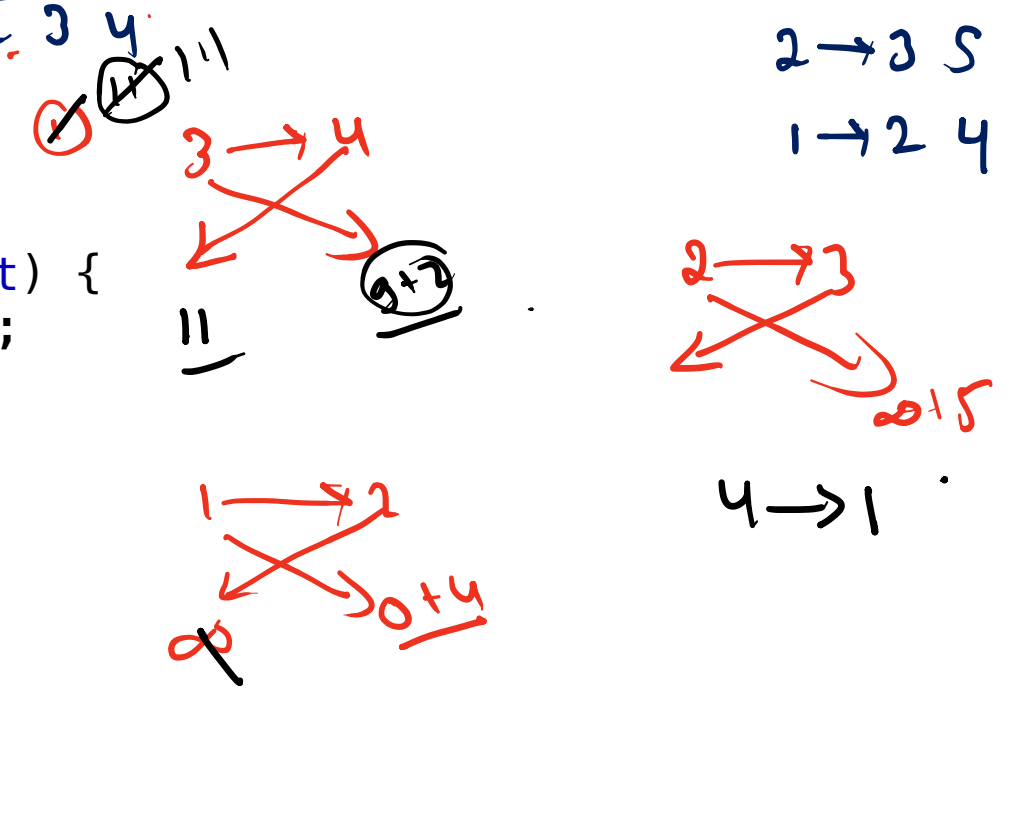


```java
public void BellManFord_Algo(int src) {
      int[] dis = new int[map.size() + 1];
      for (int i = 0; i < dis.length; i++) {
         if (i != src) {
            dis[i] = 999999;
         }
      }
      List<EdgePair> ll = getallEdeg();
      for (int i = 1; i < map.size(); i++) {
         for (EdgePair e : ll) {
            if (dis[e.e2] > dis[e.e1] + e.cost) {
               dis[e.e2] = dis[e.e1] + e.cost;
            }
         }
      }
      for (int i = 1; i < dis.length; i++) {
         System.out.println(i+" "+dis[i]);
      }
   }
```

③ —



0	1	2	3	4
	0	4	9	11

3→4 2
2→3 5
1→2 4